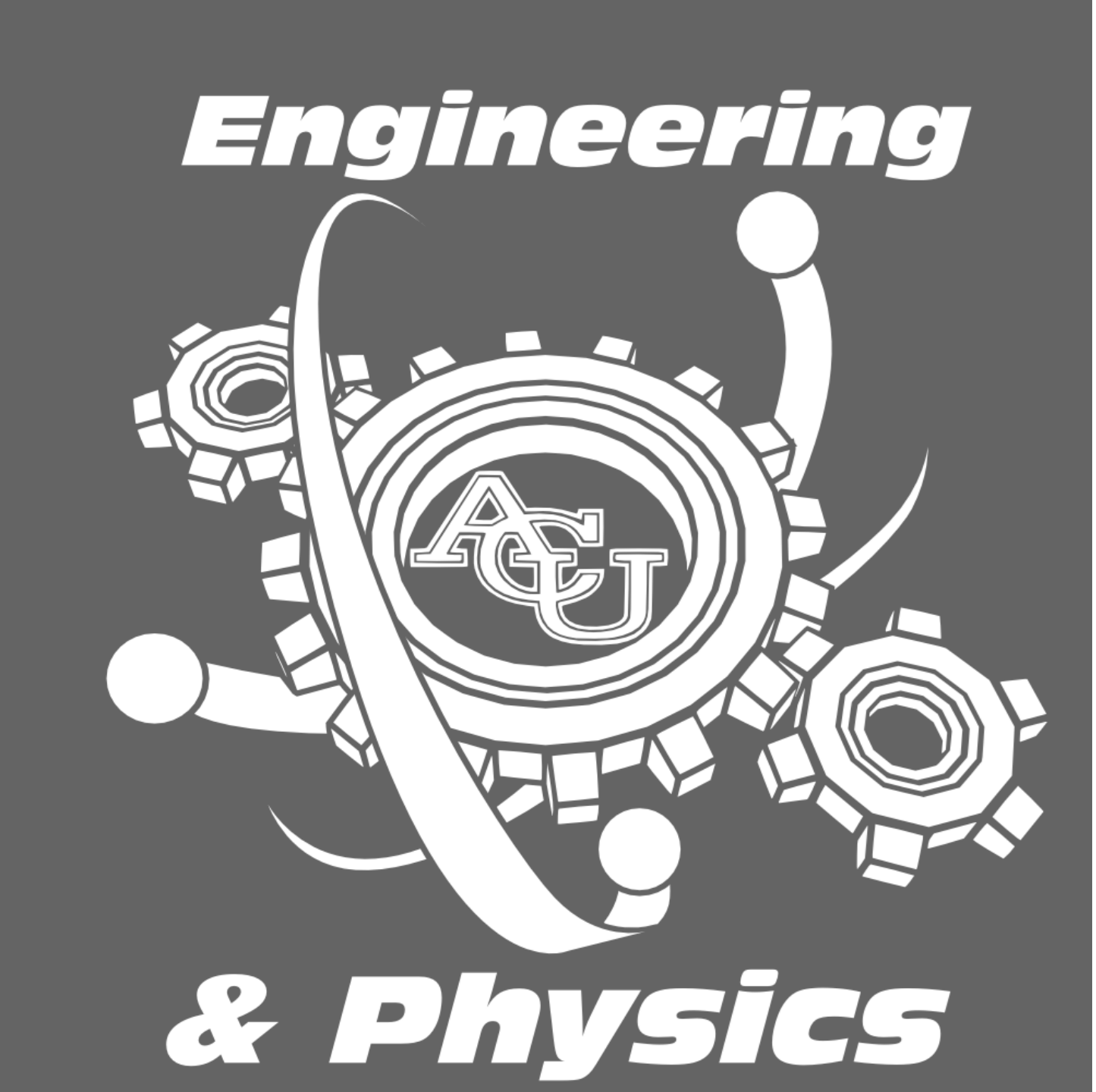


PHYS 351
Topic 2: Root Finding

Dr. Daugherity

Abilene Christian University

Problem

Solve any gross equation for x:

$$\cosh(\, x^2 e^x) = \sin x + x!$$

Problem 2

Our approach: ROOT FINDING!

Find x where $f(x) = 0$

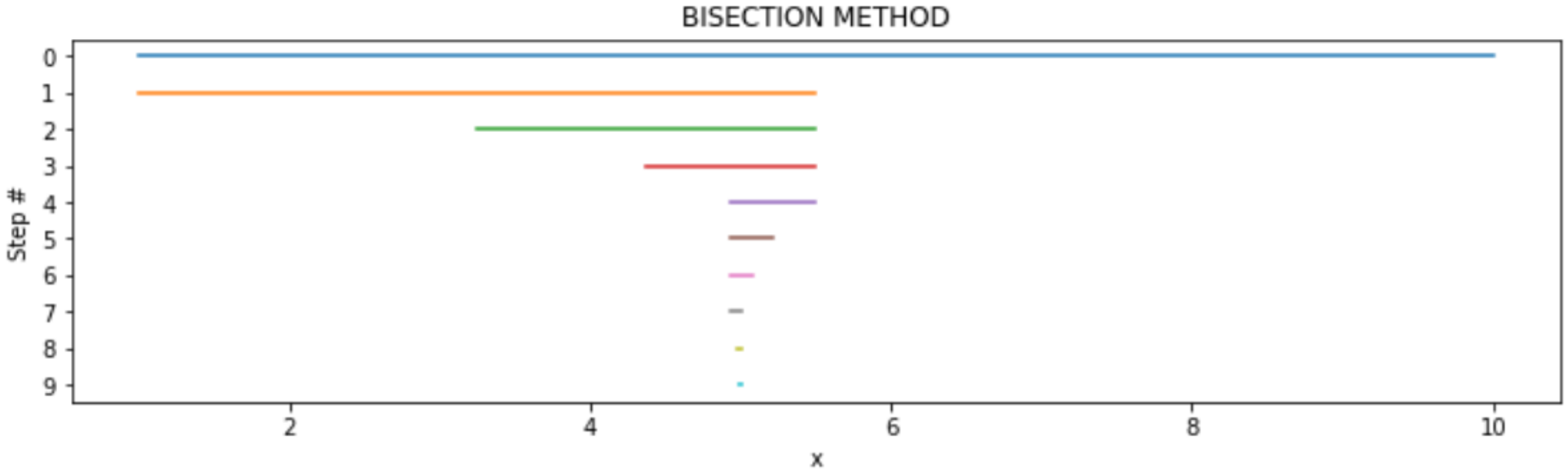
Example:

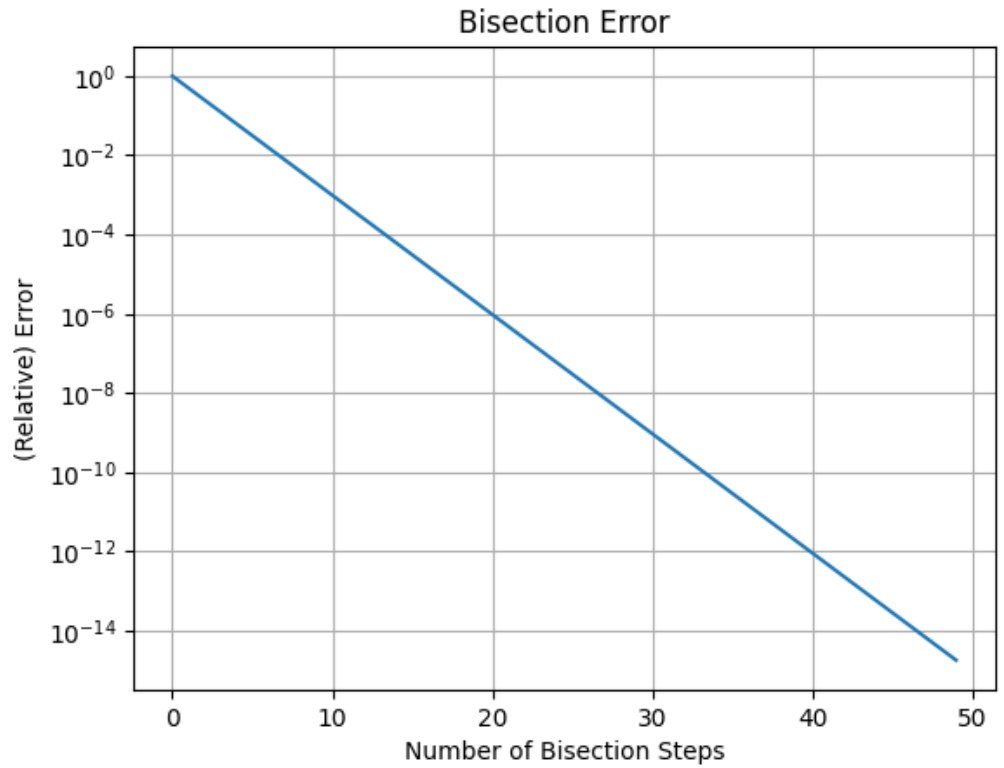
To solve: $\cosh(\, x^2 e^x) = \sin x + x!$

Find roots of

$$f(x) = \cosh(\, x^2 e^x) - \sin x + x!$$

BISECTION


BISECTION METHOD
Step #
0
1
2
3
4
5
6
7
8
9
2
4
6
8
10
x


Bisection Error
(Relative) Error
10^{0}
10^{-2}
10^{-4}
10^{-6}
10^{-8}
10^{-10}
10^{-12}
10^{-14}
0
10
20
30
40
50
Number of Bisection Steps

NEWTON-RAPHSON

https://en.wikipedia.org/wiki/Joseph_Raphson

Joseph Raphson

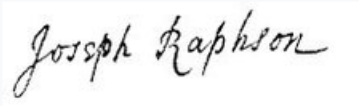
From Wikipedia, the free encyclopedia

Joseph Raphson (c. 1648 – c. 1715) was an English mathematician known best for the Newton–Raphson method.

Contents [hide]

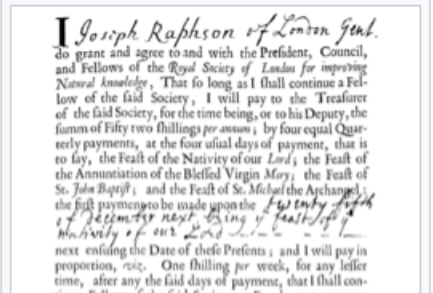
Joseph Raphson

Born	c. 1648 Middlesex, England
Died	c. 1715 England
Nationality	English
Alma mater	University of Cambridge
Known for	Newton–Raphson method
Scientific career	
Fields	Mathematician
Signature	



Biography [edit]

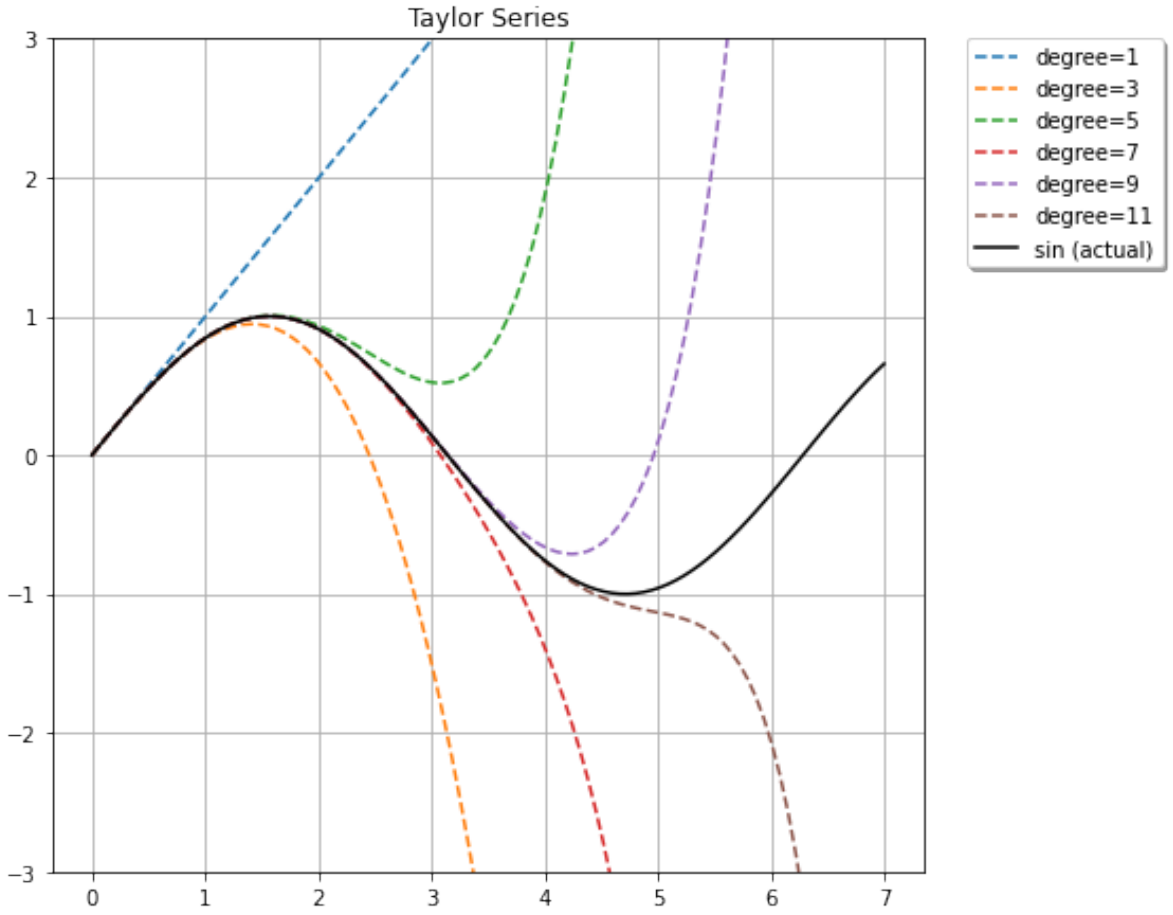
Little is known about Raphson's life, and even his exact years of birth and death are unknown, although the mathematical historian Florian Cajori provided the approximate dates 1648–1715. He was likely of Jewish and Irish descent.[1] Raphson attended Jesus College at Cambridge, graduating with an M.A. in 1692.[2] He was made a Fellow of the Royal Society on 30 November 1689, after being proposed for membership by Edmund Halley.


I Joseph Raphson of London Gent. do grant and agree to and with the President, Council, and Fellows of the Royal Society of London for improving Natural knowledge, That so long as I shall continue a Fellow of the said Society, I will pay to the Treasurer of the said Society, for the time being, or to his Deputy, the summ of Fifty two shillings per annum; by four equal Quarterly payments, at the four usual days of payment, that is to say, the Feast of the Nativity of our Lord; the Feast of the Annuntiation of the Blessed Virgin Mary; the Feast of St. John Baptist; and the Feast of St. Michael the Archangel; the first payment to be made upon the twenty fifth of December next, being y feast [illegible] of our Lord [illegible] next ensuing the Date of these Presents; and I will pay in proportion, viz. One shilling per week, for any lesser time, after any the said days of payment, that I shall con-

Raphson's most notable work is *Analysis Aequationum Universalis*, which was published in 1690. It contains a method, now known as the Newton–Raphson method, for approximating the roots of an equation. Isaac Newton had developed a very similar formula in his *Method of Fluxions*, written in 1671, but this work would not be published until 1736, nearly 50 years after Raphson's *Analysis*. However, Raphson's version of the method is simpler than Newton's, and is therefore generally considered superior. For this reason, it is Raphson's version of the method, rather than Newton's, that is to be found in textbooks today.

Taylor Series

$$f(x) \approx \ f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n$$



Practice:

1) $f(x) = \sin(x)$ for a=0

2) $\exp(x)$ for a=0

3) $f(x) = \sqrt{x}$ for a=2

Taylor Series

$$f(x) \approx \ f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n$$

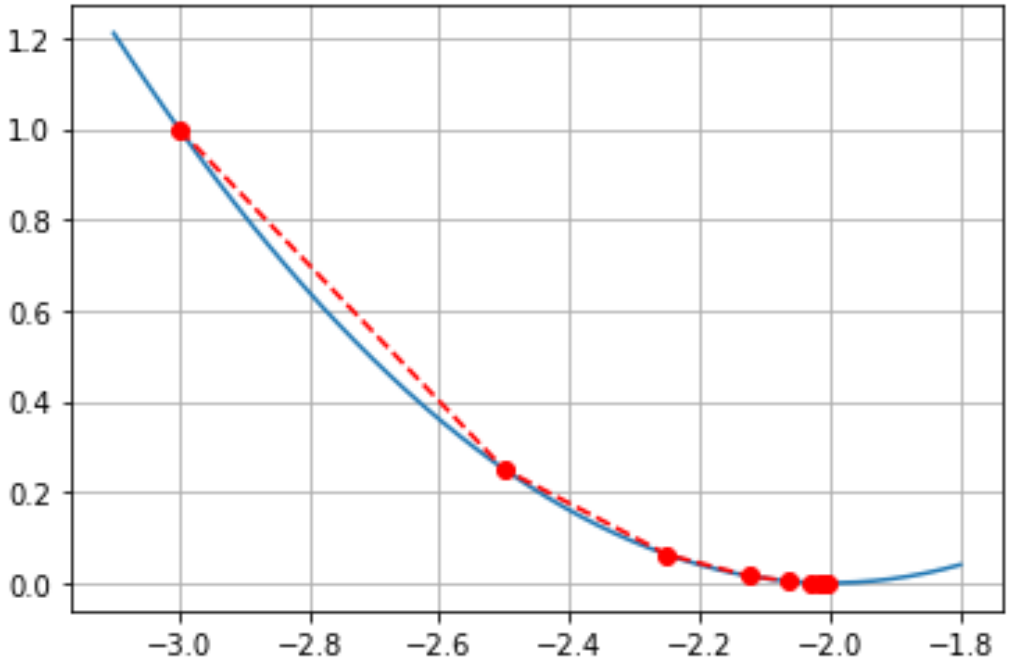
Notes:

- Truncation error = error from not including an infinite number of terms in approximations
- Any approximation like this only makes sense if each term gets smaller
- The next term provides an estimate of the truncation error

Newton-Raphson

Example: $f(x) = x^2 + 4x + 4$

```
Initial Guess =  -3
next guess is xnew = -2.5
next guess is xnew = -2.25
next guess is xnew = -2.125
next guess is xnew = -2.0625
next guess is xnew = -2.03125
next guess is xnew = -2.015625
next guess is xnew = -2.0078125
next guess is xnew = -2.00390625
```



Secant

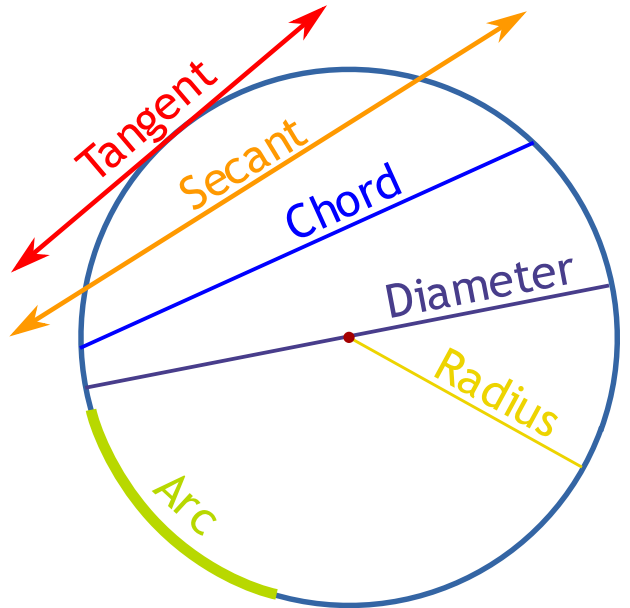
- **Problem:** The huge issue with Newton is that we need to know the derivative first. This hardly ever happens in reality.

- **Solution:** we can *approximate* the derivative using limit definitions from Cal 1:

$$f'(x_0) \approx \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

This gives us the **Secant Method**: just Newton's but with an approximate derivative

Note that we will study derivatives in details later…



Finally

ROOT_SCALAR

Method Summary

- bisection – search for the root by finding where $f(x)$ changes signs
- Newton – use the given derivative to extrapolate a straight line to the root
- Secant – just like Newton except we approximate the derivative

https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.root_scalar.html

scipy.optimize.

root_scalar

root_scalar(*f*, *args*=(), *method*=None, *bracket*=None, *fprime*=None, *fprime2*=None, *x0*=None, *x1*=None, *xtol*=None, *rtol*=None, *maxiter*=None, *options*=None) [source]

Find a root of a scalar function.

Parameters:

f : *callable*

A function to find a root of.

args : *tuple, optional*

Extra arguments passed to the objective function and its derivative(s).

method : *str, optional*

Type of solver. Should be one of

- 'bisect' (see here)
- 'brentq' (see here)
- 'brenth' (see here)
- 'ridder' (see here)
- 'toms748' (see here)
- 'newton' (see here)
- 'secant' (see here)
- 'halley' (see here)

bracket: A sequence of 2 floats, optional

An interval bracketing a root. *f(x, *args)* must have different signs at the two endpoints.

x0 : *float, optional*

Initial guess.

x1 : *float, optional*

A second guess.

fprime : *bool or callable, optional*

If *fprime* is a boolean and is True, *f* is assumed to return the value of the objective function and of the derivative. *fprime* can also be a callable returning the derivative of *f*. In this case, it must accept the same arguments as *f*.

fprime2 : *bool or callable, optional*

If *fprime2* is a boolean and is True, *f* is assumed to return the value of the objective function and of the first and second derivatives. *fprime2* can also be a callable returning the second derivative of *f*. In this case, it must accept the same arguments as *f*.

xtol : *float, optional*

Tolerance (absolute) for termination.

rtol : *float, optional*

Tolerance (relative) for termination.

maxiter : *int, optional*

Maximum number of iterations.

METHOD	INPUT	Always Converges?	Breaks if	Notes
bisection	bracket	YES	f doesn't change signs since we need opposite signed brackets	-slow and steady -sometimes hard to find bracket -fails on double roots!
Newton	derivatives, initial guess	NO	$f'(x_i) = 0$ at any time	-need equations for derivatives -FAST! -can get lost
Secant	two guesses	NO	$f(x_0) = f(x_1)$ at any time	-easiest to start -can make second guess as $x_0 + \Delta x$ -can also get lost

https://docs.scipy.org/doc/scipy/reference/optimize.html

Fancier Methods

Domain of f	Bracket?	Derivatives?		Solvers	Convergence	
		fprime	*fprime2*		Guaranteed?	Rate(s)(*)
R	Yes	N/A	N/A	• bisection • brentq • brenth • ridder • toms748	• Yes • Yes • Yes • Yes • Yes	• 1 "Linear" • >=1, <= 1.62 • >=1, <= 1.62 • 2.0 (1.41) • 2.7 (1.65)
R or *C*	No	No	No	secant	No	1.62 (1.62)
R or *C*	No	Yes	No	newton	No	2.00 (1.41)
R or *C*	No	Yes	Yes	halley	No	3.00 (1.44)

per iter. (per fun. eval)

Arguments for each method are as follows (x=required, o=optional).

method	f	args	bracket	x0	x1	fprime	fprime2	xtol	rtol	maxiter	options
bisect	x	o	x					o	o	o	o
brentq	x	o	x					o	o	o	o
brenth	x	o	x					o	o	o	o
ridder	x	o	x					o	o	o	o
toms748	x	o	x					o	o	o	o
secant	x	o		x	o			o	o	o	o
newton	x	o		x		o		o	o	o	o
halley	x	o		x		x	x	o	o	o	o

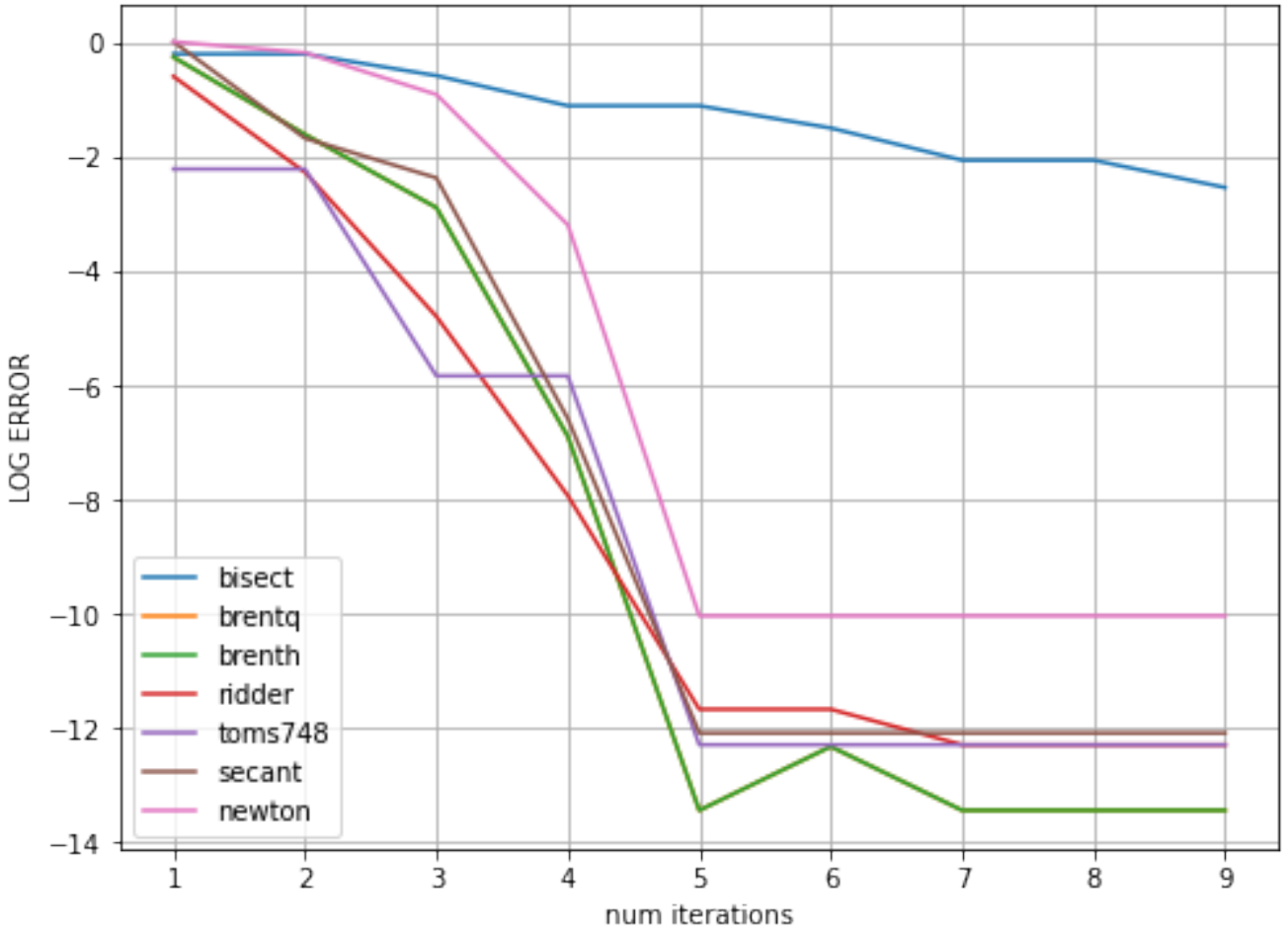
The fancier methods are bracketed to guarantee convergence

https://github.com/scipy/scipy/blob/v1.14.1/scipy/optimize/_root_scalar.py#L62

Logic for choosing method

```
246         # Pick a method if not specified.
247         # Use the "best" method available for the situation.
248         if not method:
249             if bracket:
250                 method = 'brentq'
251             elif x0 is not None:
252                 if fprime:
253                     if fprime2:
254                         method = 'halley'
255                     else:
256                         method = 'newton'
257                 elif x1 is not None:
258                     method = 'secant'
259                 else:
260                     method = 'newton'
```

The default bracketed method is brentq, which is Algorithm M in: https://dl.acm.org/doi/10.1145/355656.355659
This is a “safe” version (always bracketed) of secant using hyperbolic extrapolation



BISECTION

NEWTON

SECANT

Simple root finding method comparison. (The results are very sensitive to the problem and initial guesses)

- Bisection converges very slowly
- Newton is still bad
- The fancier bisection methods are fastest, but secant is almost as good

Dr. D's Advice

- Plot the function
- Use brackets to know which root you're getting
- If you just want an answer, use secant (x0=initial guess, x1=x0+1e-4), but know that you might not get the root closest to x0

Practice

Find all solutions where $x > 0$ of the equation:

$$\sin x = 0.1\, x$$

Practice

The thermodynamic efficiency η of a certain engine is given by:

$$\eta = \frac{\ln(T_2/T_1) - (1 - T_1/T_2)}{\ln(T_2/T_1) + (1 - T_1/T_2)/(\gamma - 1)}$$

where $\gamma = \frac{5}{3}$. Find the temperature ratio $\frac{T_2}{T_1}$ where $\eta = 0.3$

(Be careful with the fractions here…)

Non-Linear Systems

We can handle systems of equations in a similar way using **root** instead of **root_scalar**

scipy.optimize.root(*fun, x0, args=(), method='hybr', jac=None, tol=None, callback=None, options=None*) [source]

Find a root of a vector function.

Parameters: **fun** : *callable*

A vector function to find a root of.

x0 : *ndarray*

Initial guess.

args : *tuple, optional*

Extra arguments passed to the objective function and its Jacobian.

```
1 from scipy.optimize import root
```

```
1 def f(x):
2   return [x[0]-3, x[1]-4]
```

```
1 root(f, [1,2])
```

```
   fjac: array([[-1.00000000e+00, -1.09079412e-12],
          [ 1.09079412e-12, -1.00000000e+00]])
    fun: array([0., 0.])
message: 'The solution converged.'
   nfev: 5
    qtf: array([-4.36228831e-12, -4.36273240e-12])
      r: array([-1.00000000e+00, -2.18131069e-12, -1.00000000e+00])
 status: 1
success: True
      x: array([3., 4.])
```

Carl Gustav Jacob Jacobi
1804 - 1851

Jacobian

https://en.wikipedia.org/wiki/Jacobian_matrix_and_determinant

Definition [edit]

Let $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ be a function such that each of its first-order partial derivatives exists on $\mathbb{R}^n$. This function takes a vector $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{R}^n$ as input and produces the vector $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_m(\mathbf{x})) \in \mathbb{R}^m$ as output. Then the Jacobian matrix of $\mathbf{f}$, denoted $\mathbf{J}_\mathbf{f}$, is the $m \times n$ matrix whose (i, j) entry is $\frac{\partial f_i}{\partial x_j}$; explicitly

$$\mathbf{J}_\mathbf{f} = \begin{bmatrix} \dfrac{\partial \mathbf{f}}{\partial x_1} & \cdots & \dfrac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^\mathsf{T} f_1 \\ \vdots \\ \nabla^\mathsf{T} f_m \end{bmatrix} = \begin{bmatrix} \dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial f_m}{\partial x_1} & \cdots & \dfrac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Example 2: polar-Cartesian transformation [edit]

The transformation from polar coordinates (r, φ) to Cartesian coordinates (x, y), is given by the function $\mathbf{F}$: $\mathbf{R}^+ \times [0, 2\pi) \to \mathbf{R}^2$ with components

$$\begin{aligned} x &= r\cos\varphi; \\ y &= r\sin\varphi. \end{aligned}$$

$$\mathbf{J}_\mathbf{F}(r, \varphi) = \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \varphi} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \varphi} \end{bmatrix} = \begin{bmatrix} \cos\varphi & -r\sin\varphi \\ \sin\varphi & r\cos\varphi \end{bmatrix}$$

The Jacobian determinant is equal to r. This can be used to transform integrals between the two coordinate systems:

$$\iint_{\mathbf{F}(A)} f(x, y)\, dx\, dy = \iint_A f(r\cos\varphi, r\sin\varphi)\, r\, dr\, d\varphi.$$

Things to Know

- Why root finding lets us solve for x
- How to use `root_scalar` in 3 different ways
- Jacobian = matrix of derivatives
- How to do multi-dimensional cases